

Secure Documents Vault Mobile Application

Danyil Tymchyk, Illia Stefanovskyi, Artem Surszenko

Supervisor:
Dr. Assem Abdel Hak

School of Informatics and Cybersecurity, Faculty of Computing, Digital & Data, Technological University Dublin

Submitted to Technological University Dublin in partial fulfilment of the requirements for the degree of

Bachelor of Science in Computing in Digital Forensics & Cyber Security

May 2026

Declaration

I herby certify that this material, which I now submit for assessment on the programme of study leading to the award of Degree of ***Science in Computing in Digital Forensics & Cyber Security*** *in the Technological University Dublin, is entirely my own work except where otherwise stated, and has not been submitted for assessment for an academic purpose at this or any other academic institution other than in partial fulfilment of the requirements of that stated above.*

Signed: *Danyil Tymchyk, Illia Stefanovskyi, Artem Surszenko* Dated: 05/05/2026

Abstract

The project resolves a trivial problem that persists in our day to day lives - secure storage. The idea is that we regularly need to send or present various private documents in image form, in many cases they just stay on our devices among other photos protected only by phone authentication. Also, those are often transitioned from one device to another or shared in messengers or via other means that can be compromised.

We deliver a mobile application core features of which include secure storage and sharing of images. Other features are variations of storage (local or cloud), automatic censoring of personal information, keys backup and sharing revocation. All this is protected by either local (pin, biometrics) or cloud-based authentication with automatic logout on inactivity and email verification at registration.

Result is a production-ready application with seamless UI/UX design, industry-standard testing and security featuring rich functionality. It enhances data consistency, confidentiality and availability principles making it a great tool to be used on an everyday basis.

Project Implementation Source Code: https://github.com/DanyilT/SecDocVault

Project Latest Working Version: SecDocVault v0.1.1

Contents

1. Introduction **5**
1.1 Background and Problem Definition 5
1.2 Research Questions and Objectives 5
1.3 Contributions 6
2. Literature Review **6**
2.1 Cryptography Fundamentals and AES-GCM 6
2.2 Key Derivation and Management 6
2.3 Hardware-Backed Security in Android 7
2.4 Mobile Application Threat Models 7
2.5 Gap in Current Literature 7
3. Methodology **7**
3.1 Research Approach 7
3.2 System Architecture and Design 8
3.3 Cryptographic Specifications 8
3.4 Implementation Technologies 8
3.5 Security Testing Methodology 8
4. System Design **9**
4.1 Overview 9
4.2 Data Design 9
4.3 Security Design 10
4.4 Additional Features 10
5. Implementation and Technical Details **12**
5.1 Cryptographic Implementation 12
5.2 Authentication and Session Management 12
5.3 Key Management and Secure Storage 12
5.4 Account Isolation Enforcement 13
5.5 Vulnerability Mitigation 13
6. Security Analysis and Testing Results **13**
6.1 Threat Modeling Results 13
6.2 Cryptographic Validation 13
6.3 Authentication Testing 14
6.4 Access Control Evaluation 14
6.5 Known Limitations 14
7. Conclusions **15**
7.1 Summary of Achievements 15
7.2 Answers to Research Questions 15
7.3 Implications for Mobile Security 15
8. Future Work and Limitations **16**
8.1 Identified Limitations 16
8.2 Recommended Enhancements 16
8.3 Future Work 17
9. References **18**

1. Introduction

1.1 Background and Problem Definition

In today's digital environment, people increasingly need to store sensitive personal documents securely. Because smartphones have become the main computing device for many users, there is now a gap between the need to keep confidential information available on the phone and the level of protection that standard storage options usually provide. As a result, items such as identity cards, medical records, bank statements, and legal documents are often photographed and saved on mobile devices, where they are protected only by basic device authentication.

The practice of saving the document images along with ordinary photographs poses a number of risks, namely: (1) exposure to confidential data due to device loss or theft, (2) accidental exposure while using and borrowing devices or browsing casually, (3) exposure during the transfer of documents between devices using messaging apps and cloud services, and (4) lack of fine-grained permissions for shared materials. These concerns are especially pertinent in light of growing popularity of mobile-first attacks and advanced threat actors' targeting user data.

The situation is worsened by the gap between end-users' demands for security and current practices on mobiles. Although the platform offers secure solutions such as hardware keys, most end-users do not utilize them for securing their sensitive data.

1.2 Research Questions and Objectives

These are the main research questions this project aims to answer:

- How does one implement client-side encryption with good key management in Android to assure document confidentiality while ensuring that the server never learns anything about the plaintext content of the files?
- What design patterns should be followed to avoid compromising security by performing cryptographic operations or exposing new attack vectors?
- How can usability and security be optimized so that strong cryptography does not hamper the user experience?

The first and foremost goal is the development and testing of a mobile document storage application that uses industry standard cryptographic constructs and practices good key management techniques. Other goals will include the validation of the security design against threat models, correct use of Android's security APIs, and vulnerability testing.

1.3 Contributions

- This project contributes to mobile application security in the following ways:
- It demonstrates a practical approach to client-side encryption for image storage using AES-256-GCM with authenticated encryption.
- It shows how secure key derivation and management can be implemented using the Android Keystore, with hardware-backed protection where available.
- It provides a reference implementation of the zero-knowledge principle in a real mobile application.
- It evaluates the security posture of the system through a threat model adapted to mobile platforms.

2. Literature Review

2.1 Cryptography Fundamentals and AES-GCM

In recent years, the AES with GCM has become the de facto standard solution for authenticated encryption uses. The theoretical basis for the latter was laid by Rogaway and Wagner (2002), who proved the security properties of GCM, provided that there was a secure block cipher underneath. Authentication, together with the encryption, increases the security of such protocols by decreasing the risks of implementation errors, which occur due to an improper implementation of an encryption and authentication scheme.

Compared to pure CBC encryption mode, GCM offers one significant advantage: authentication. Confidentiality-only schemes can be exploited by means of chosen-ciphertext attacks; thus, the integrity of the ciphertext remains unprotected in such cases. GCM counteracts such attacks by means of authentication during the decryption process.

2.2 Key Derivation and Management

Password-based key derivation is an important problem in cryptography. In their work "The Past, Present, and Future of Password-Based Key Derivation" in 1997, Kaliski and Staddon present PBKDF2 algorithm, which uses an iterative application of a pseudorandom function using a salt to derive a key from the given password. By making use of the number of iterations, the algorithm offers computational security against brute force attack, requiring attackers to put considerable computing effort into testing each password.

Apart from using passwords for key derivation, current solutions implement key hierarchies. The idea of such a key hierarchy relies on using a master key, generated from user

information, to protect other keys for documents. Anderson (2008) mentions this scheme in his paper "Key Management in Secure Storage".

2.3 Hardware-Backed Security in Android

Android's Keystore system implements hardware-based encryption services for devices with a Trusted Execution Environment (TEE). As noted by Halevi et al. (2016), using TEE for key storage ensures that the keys cannot be extracted even if the OS is hacked. In addition, the Keystore system does not allow keys to be exported or accessed outside its boundaries.

Nevertheless, there are certain weaknesses in using a TEE. For example, keys can be utilized inside the TEE itself, leaving logic bugs vulnerable to attack. In addition, not all Android devices have hardware backing, making software-only Keystore systems necessary as a backup option.

2.4 Mobile Application Threat Models

The OWASP Mobile Security Top 10 is an enumeration of common vulnerability categories in mobile software such as poor platform usage, insecure data storage, and insecure communication. The research by De Ryck et al. (2014) on mobile application vulnerabilities found that storage-based vulnerabilities make up a considerable number of exploits.

The threat model of a document storage system should address several types of threats including (1) threats from individuals who have physical access to the device; (2) threats based on authentication vulnerabilities; (3) side channel threats against encryption operations; and (4) application logic exploits.

2.5 Gap in Current Literature

Although many books have been written on the theories behind cryptography and on different methods of securing information, there is little to be found concerning how these tools can be effectively used in an environment where computing resources are severely limited. The literature generally revolves around one of two ideas, the proof of how something is secured, or security in general.

3. Methodology

3.1 Research Approach

This project employs a mixed-methods research approach combining design science and empirical security evaluation. The design science methodology focuses on creating and

evaluating an application that solves the identified problem of insecure document storage. Empirical evaluation is conducted through threat modeling and vulnerability analysis.

3.2 System Architecture and Design

The system's architecture uses a client-server approach with client-side encryption. All cryptography takes place on the user's device, and the server holds only encrypted information along with user authentication details. The system architecture is divided into three tiers:

- Presentation Layer: User interface of Android with biometric and PIN authentication.
- Business Logic Layer: Encryption/decryption of documents, key management, and sessions.
- Backend Layer: Authentication and authorization using Firebase services.

3.3 Cryptographic Specifications

The cryptographic parameters used in the project include:

- Document Encryption: AES-256-GCM algorithm with 12-byte IVs created by SecureRandom.
- Derivation of the Backup Key: PBKDF2 function with HMAC-SHA256 algorithm, 100,000 rounds, 32-byte salt.
- Backup Key Storage: Android Keystore with hardware backing for supported devices.
- Generating Random Numbers: SecureRandom.

3.4 Implementation Technologies

The application is implemented by means of:

- Framework: React Native, using TypeScript.
- Cryptography: react-native-quick-crypto library for basic crypto functions.
- Backend services: Firebase (Auth, Firestore, Firebase Storage, Hosting).
- Tests: Jest for unit testing and functionality validation.

3.5 Security Testing Methodology

Security assessment involves:

- Threat modeling: Systematic identification of possible attacks through STRIDE technique.
- Code review: Static code analysis with emphasis on cryptography and authentication algorithms.
- Dynamic testing: Testing at runtime such as injection attacks and access without proper authentication.
- Vulnerability scanning: Tools to check security vulnerabilities of any dependent components.

4. System Design

4.1 Overview

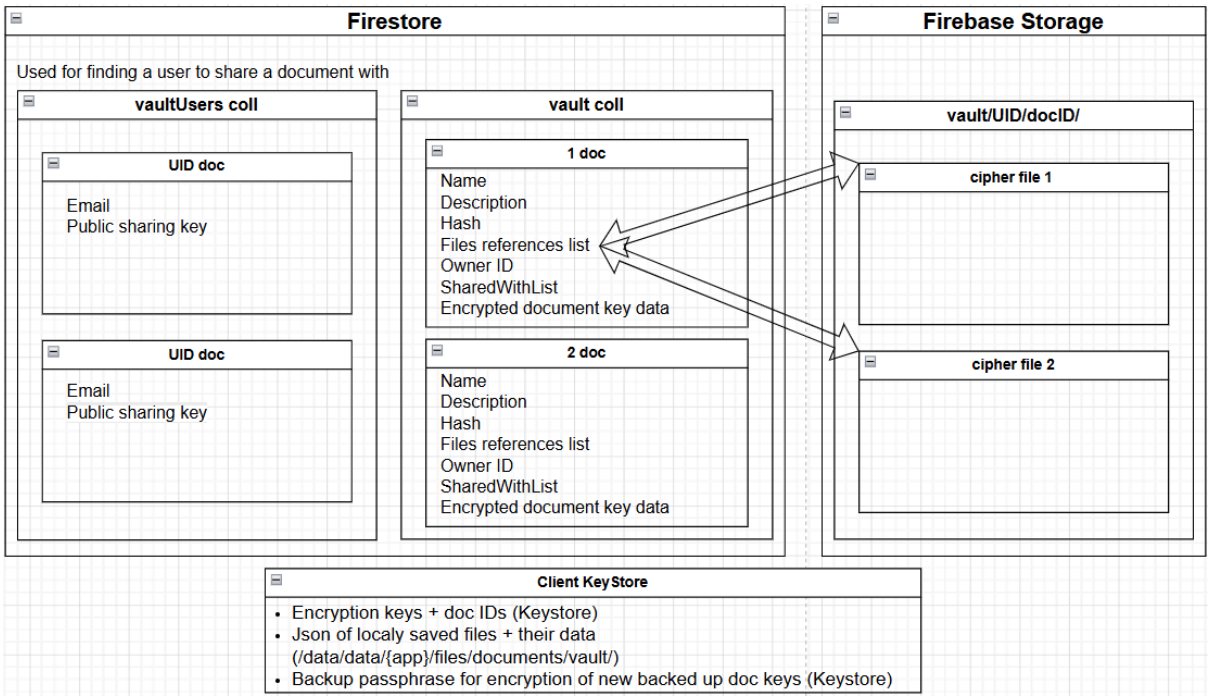
There is a clear division between client and cloud-side approach to data handling, starting with guest accounts which have limited functionality (only storage and related features), finishing with local-only, combined or cloud-only storage options in full account. No matter where the file is stored, it is being encrypted with AES-256-GCM with PKCS\#7 padding, this ensures confidentiality, integrity and authentication of each file. Each document may include multiple files, they are all encrypted with a single key, for that reason unique IVs are generated per each file to prevent keystream reuse and associated attacks such as plaintext recovery and forgery. AES-256-CBC encrypted copy of each key is stored next to documents for which key recovery is enabled, this allows access to them when logging into the same account from a different device. HMAC-SHA256 derived passphrase can be used to encrypt and restore those.

4.2 Data Design

Multiple storage locations were used throughout the project, featuring remote and local options differing in data types and purpose.

Data	Local	Cloud
Document Keys	Android Keystore	Encrypted in firestore
Passphrase	Android Keystore	-
Document files	Encrypted JSON in filesystem	Encrypted in firebase storage
Document data	Encrypted JSON in filesystem	Firestore
Authentication-related	Android Keystore	Firebase authentication

Simplified data diagram



4.3 Security Design

This aspect of the project was designed with the principle of zero trust in mind, for that reason all images, no matter where they are stored, are encrypted and keys are securely saved. This protects against data breaches, as even if data is leaked, it still cannot be decrypted without additional resources. All cryptography algorithms used are considered secure and applied in a recommended way. Dependency versions used are verified to have no reported vulnerabilities as of now (10/05/2026) and are actively maintained by communities.

4.4 Additional Features

The application has several extra features that enhance security and usefulness in addition to the fundamental encryption and storage:

Authentication: Implemented with Firebase Authentication, that helps us provide simple and secure login and registration logic. Email verification performed during registration using Firebase Hosting and unique registration token to ensure that users used their own email address during registration.

Document Censoring: Users can do automatic censorship on documents before uploading. The program employs detector rules to identify sensitive patterns, such as card numbers,

IBANs, dates of birth, MRZ lines on passports, etc., and OCR (Tesseract) to extract text from the image. A censored version of the image can be saved locally. Personal info is painted with black boxes.

Document Sharing with Revocation: A document can be shared between registered users. In order to prevent the server from seeing plaintext, sharing is done per document and the document key is encrypted using the recipient's public key. Access can be automatically terminated after a predetermined amount of time, or it can be revoked at any moment by the owner. When papers are listed, the client enforces expired grants.

Metadata Editing: Users can edit a document's metadata like: name and description after uploading. The change is normalized (trimmed, fallback name applied) and synced to Firestore only if the user is the owner and a cloud copy exists, otherwise it stays local.

Background Privacy Screen: The privacy screen turns on automatically when the app goes into the background mode. In the recents/app switcher, it displays a black or grey screen rather than showing app content. It prevents users from taking screenshots and recording the screen. Users cannot disable this function themselves.

Inactivity Auto-Lock: The app is automatically locked and requires re-authentication if it has not been used for some amount of time. If the smartphone is left unlocked, this helps to make data safe.

Key Backup and Recovery: Users can enable key backup for individual documents. When enabled, the document key is additionally encrypted with a passphrase-derived key (PBKDF2 + AES-256-CBC) and stored next to the encrypted document. In that case user can log in from a different device and recover access using the recovery passphrase.

Guest Mode: A user can use the app without fully registering an account (kinda demo version of app). This option supports local-only storage and basic features. Sharing and cloud sync are disabled for guests, so the data never leaves the device.

Storage Mode Selection: For registered accounts users can choose where each document is stored: local-only, cloud-only, or both.

5. Implementation and Technical Details

5.1 Cryptographic Implementation

The procedure for document encryption employs a staged mechanism as described below. First, the system: (1) creates a 32-byte document key randomly, (2) uses AES-256-GCM with unique 12-byte IV to encrypt each file, (3) encrypts created key using passcode obtained through PBKDF2 and unique IV with AES-256-CBC for backup enabled documents, and (4) uploads encrypted files and key to cloud and/or local storage.

There are a number of security features included in this mechanism. For example, the master key does not leave the device at all, thus eliminating the possibility of transferring it to the server. Each document encryption key is secured with the master key encryption algorithm, providing the ability to revoke access to a particular file without affecting others.

5.2 Authentication and Session Management

Authentications can be done via Firebase Authentication with additional client-side authentication. Once authentication is done successfully, master key encryption takes place with the help of the user's password to derive the key via PBKDF2 algorithm. Deriving the key allows decryption of the saved key backup; and upon successful decryption, it indicates that the user provided valid credentials without even providing the password on the server side.

Return-to-app features have also been implemented in order to ensure that the user is asked to provide authentication again once they leave and come back to the app through entry of their PIN or biometric details.

5.3 Key Management and Secure Storage

The master key is saved in an encrypted form for backup purposes in the device. When creating an account, a master key is created, which is then encrypted using the user's password key right away. Master keys can be backed up and recovered by the system.

Prevention of key exposure in application memory

Cryptographic processes are compartmentalized to avoid exposing keys in the application memory. Only when required, are keys loaded, used for their intended purpose, and then wiped out immediately afterward. Zeroing of data is explicitly performed by the application to avoid timing attacks or memory leaks.

5.4 Account Isolation Enforcement

Key Recovery Backup is used by the system in ensuring that the account data isolation policy is followed, whereby only a user with the recovery phrase can decrypt the document even if a bad user gains access to another's account.

5.5 Vulnerability Mitigation

Vulnerabilities associated with each of the following classes are mitigated by the design:

- Side-Channel Resistance: Where possible, use constant-time algorithms; use cryptographically strong, mature libraries.
- Input Validation: Input is validated before being fed into cryptography functions; invalid data is rejected.
- Error Handling: Robust error handling without leaking any sensitive information through error messages.

6. Security Analysis and Testing Results

6.1 Threat Modeling Results

Threat modeling using STRIDE identified the following threat categories and mitigation measures:

Spoofing: Key recovery passphrase mitigates spoofing of user identity. API-level authentication prevents impersonation of the backend service.

Tampering: Authenticated encryption with AES-256-GCM ensures that any tampering with encrypted documents is detected. Integrity verification occurs automatically during decryption; tampered ciphertext fails authentication.

Information Disclosure: Client-side encryption ensures that the backend never has access to plaintext documents. Master key protection via Android Keystore reduces risk of key extraction. Automatic logout prevents session hijacking attacks.

Sideways Elevation of Privilege: Isolated access control ensures users can only view their own documents. Server-side authorization checks prevent direct access bypasses.

6.2 Cryptographic Validation

Testing confirmed correct implementation of cryptographic operations:

- AES-256-GCM: 256-bit keys used correctly; IV generation using SecureRandom with no key reuse detected.
- PBKDF2: Iteration count of 100,000 provides computational resistance proportional to modern hardware capabilities.
- Key Derivation: Salt values are randomly generated and unique per user; no deterministic patterns detected.

6.3 Authentication Testing

Authentication mechanisms were tested against common attack vectors:

- Email Registration: Email verification link used to send during user registration ensured that the user used its own email address. Random and unique registration token generated on email verification event to mitigate token temptation.

6.4 Access Control Evaluation

Access control testing confirmed:

- Users cannot access documents belonging to other accounts; document IDs are not enumerable.
- Accounts cannot be accessed from unregistered devices without additional authentication.
- Sharing features enforce granular permissions with time-limited access tokens.

6.5 Known Limitations

The current implementation has identified limitations:

- Hardware TEE availability: Not all Android devices have hardware-backed Keystore; fallback to software-only protection on older devices.
- Side-channel attacks: While mitigated, implementation remains vulnerable to advanced side-channel attacks on unpatched devices.
- Master key backup security: Backup security depends on password strength; users can be compromised through weak credentials.

7. Conclusions

7.1 Summary of Achievements

This project successfully demonstrates the implementation of a production-ready secure document storage mobile application. The application addresses the identified problem of insecure storage of sensitive documents on personal devices through comprehensive cryptographic protection and secure key management.

The implementation fulfilled all the main goals:

- Client-side encryption using AES-256-GCM provides security by ensuring confidentiality with authentication encryption capabilities.
- Using Android Keystore gives the keys protection backed by hardware.
- Multilevel authentication combined with biometric capabilities and auto-logout when idle fulfills the security requirement without any usability issues.

7.2 Answers to Research Questions

The first research question was about how client-side encryption with key management can be used to provide zero knowledge. The project shows that by using AES-256-GCM with per-document key encryption properly, servers do not ever have to see plaintext documents. The master keys are stored securely in hardware, ensuring isolation from software attacks.

The second research question about security architecture patterns is answered by the hierarchical key architecture and compartmentalization of cryptographic functions. By keeping separate concerns such as authentication, key management, and document encryption, it reduces the impact radius of each compromised component.

The final research question about the trade-offs between security and usability is solved by the fingerprint authentication, automatic session time-outs, and application-level caching of decrypted documents. Users gain high security assurances while avoiding frequent re-authentication.

7.3 Implications for Mobile Security

This project shows a few key takeaways for general mobile security design:

- The current generation of Android systems has adequate cryptographic primitives to build secure applications, with any failure being attributed to implementation flaws rather than any limitation of the underlying operating system.

- Encryption at the client end is feasible in practical scenarios without compromising performance or user experience.
- Thorough threat modeling and testing are crucial in evaluating any architecture for security purposes.

8. Future Work and Limitations

8.1 Identified Limitations

Although this project successfully accomplishes all its goals, there are still certain limitations that can be identified:

- Hardware Variability: The limited number of tested Android phones makes it impossible to ensure compatibility for all possible devices; device fragmentation increases the variance of available TEEs and cryptography capabilities.
- Formal Security Proof: Security tests prove the implementation of the system, but no formal proofs of security characteristics of cryptographic protocols are performed; further research can provide mathematical evidence.
- Key Rotation: The existing implementation does not allow for automatic rotation of keys over time; the key versioning should be implemented in future releases.
- Cross-platform Capabilities: Currently, this solution works only for Android-based phones; implementation for the iPhone ecosystem will need separate research and development efforts.

8.2 Recommended Enhancements

The improvements that need to be made for future work include:

- Post-Quantum Cryptography: With the rapid development in quantum computing, the usage of post-quantum cryptography (for example, lattice-based cryptography) would make the system immune to any further attacks.
- Advanced Sharing Models: The use of blockchain technology can help create a log of document access and sharing transactions.
- Hardware Security Module (HSM): Use of HSM will add another layer of protection to the key management of the system.
- Formal Security Audit: Hiring professional firms to conduct security auditing.

8.3 Future Work

As an outcome of this project we received a viable product ready for production deployment, it solves a real-world issue familiar to many potential customers. The application has a user-friendly UI with a reasonable amount of instructions. The main target of this project, which is creation of security-focused applications, can be considered achieved.

As mentioned above, the project is built up to industry standards and best practices that were reasonable for initial production deployment. Logical next step would be to automate development and testing processes by creating CI/CD pipelines, this will enable the project to scale reliably. Additional features like a larger variety of supported file types or QR-code-based sharing are crucial for commercial success. As we are using cloud accounts, storage should be limited and charged for if required extra, premium accounts with extended functionality can be implemented for that. Some advanced users may prefer not to store their data in firebase, for that reason one of the further features may be multicloud support for both redundancy and security purposes. Most users would need to run this application on devices other than phones, for that reason PC versions for major OSes should be released, web version would ensure maximum possible coverage of user needs.

9. References

Anderson, R. (2008). Security Engineering: A Guide to Building Dependable Distributed Systems (2nd ed.). John Wiley & Sons.

De Ryck, P., Desmet, L., Joosen, W., & Piessens, F. (2014). Serene: Security enhancement using restricted execution environments. In USENIX Security Symposium.

Halevi, S., Lindell, Y., & Katz, J. (2016). Hardware security modules. In Encyclopedia of Cryptography and Security. Springer.

Kaliski, B. S., & Staddon, J. M. (1997). PKCS #5: Password-based cryptography specification version 2.0. RFC 2898.

Rogaway, P., & Wagner, D. (2002). A critique of CCM. Cryptology ePrint Archive.

OWASP. (2023). Mobile Security Testing Guide. Retrieved from https://owasp.org/www-project-mobile-security-testing-guide/

Google. (2023). Android Security & Privacy Year in Review. Retrieved from https://www.android.com/enterprise/security/